

DSA Lab Assignment – 7 (08/12/25 to 10/12/25)

1. Write a C program to create a binary search tree and display the content of each node.

```
#include <stdio.h>

#include <stdlib.h>

struct node {
    int info;
    struct node *left;
    struct node *right;
};

typedef struct node* nodeptr;

/* Create a new node */
nodeptr getnode() {
    return (nodeptr)malloc(sizeof(struct node));
}

/* Make a single-node tree */
nodeptr maketree(int x) {
    nodeptr p = getnode();
    p->info = x;
    p->left = NULL;
    p->right = NULL;
    return p;
}
```

```

/* Insert a new key into the BST */
nodeptr insert(nodeptr root, int x) {
    nodeptr p, q;

    if (root == NULL)
        return maketree(x);

    p = q = root;

    /* Find the proper place to insert */
    while (q != NULL) {
        p = q;
        if (x < p->info)
            q = p->left;
        else if (x > p->info)
            q = p->right;
        else {
            /* duplicate – just ignore or print a message */
            printf("%d is a duplicate value\n", x);
            return root;
        }
    }

    /* Attach new node as left or right child of p */
    if (x < p->info)
        p->left = maketree(x);
    else
        p->right = maketree(x);
}

```

```

    return root;
}

/* Inorder traversal – L, Root, R */
void inorder(nodeptr p) {
    if (p != NULL) {
        inorder(p->left);
        printf("%d\t", p->info);
        inorder(p->right);
    }
}

/* Preorder traversal – Root, L, R */
void preorder(nodeptr p) {
    if (p != NULL) {
        printf("%d\t", p->info);
        preorder(p->left);
        preorder(p->right);
    }
}

/* Postorder traversal – L, R, Root */
void postorder(nodeptr p) {
    if (p != NULL) {
        postorder(p->left);
        postorder(p->right);
        printf("%d\t", p->info);
    }
}

```

```

    }
}

int main() {
    nodeptr root = NULL;
    int n, x, i;

    printf("How many nodes? ");
    scanf("%d", &n);

    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        root = insert(root, x);
    }

    printf("\nInorder traversal (sorted order):\n");
    inorder(root);

    printf("\n\nPreorder traversal:\n");
    preorder(root);

    printf("\n\nPostorder traversal:\n");
    postorder(root);

    printf("\n");
    return 0;
}

```

Output:

How many nodes? 7

Enter 7 integers:

50 30 70 20 40 60 80

Inorder traversal (sorted order):

20 30 40 50 60 70 80

Preorder traversal:

50 30 20 40 70 60 80

Postorder traversal:

20 40 30 60 80 70 50

2. Write a C program to implement three tree traversal methods.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left;
    struct node *right;
};

typedef struct node* NODE;

/* Create a new node */
NODE createNode(int x) {
    NODE p = (NODE)malloc(sizeof(struct node));
    p->data = x;
    p->left = NULL;
    p->right = NULL;
    return p;
}
```

```

/* Insert into Binary Search Tree */
NODE insert(NODE root, int x) {
    if (root == NULL)
        return createNode(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else if (x > root->data)
        root->right = insert(root->right, x);

    return root;
}

/* Inorder Traversal (L, Root, R) */
void inorder(NODE root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d\t", root->data);
        inorder(root->right);
    }
}

/* Preorder Traversal (Root, L, R) */
void preorder(NODE root) {
    if (root != NULL) {
        printf("%d\t", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

/* Postorder Traversal (L, R, Root) */
void postorder(NODE root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d\t", root->data);
    }
}

int main() {
    NODE root = NULL;
    int n, x, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        root = insert(root, x);
    }

    printf("\nInorder traversal:\n");

```

```

inorder(root);

printf("\n\nPreorder traversal:\n");
preorder(root);

printf("\n\nPostorder traversal:\n");
postorder(root);

return 0;
}

```

Output:

```

Enter number of nodes: 7
Enter 7 elements:
50 30 70 20 40 60 80

Inorder traversal:
20 30 40 50 60 70 80

Preorder traversal:
50 30 20 40 70 60 80

Postorder traversal:
20 40 30 60 80 70 50

```

3. Write a C program to sort an array of n elements using Binary tree sort technique.

```

#include <stdio.h>
#include <stdlib.h>

/* Definition of tree node */
struct node {
    int data;
    struct node *left;
    struct node *right;
};

typedef struct node* NODE;

/* Create a new node */
NODE createNode(int x) {
    NODE p = (NODE)malloc(sizeof(struct node));
    p->data = x;
    p->left = NULL;
    p->right = NULL;
    return p;
}

/* Insert element into Binary Search Tree */

```

```

NODE insert(NODE root, int x) {
    if (root == NULL)
        return createNode(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else
        root->right = insert(root->right, x);

    return root;
}

/* Inorder traversal gives sorted order */
void inorder(NODE root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d\t", root->data);
        inorder(root->right);
    }
}

int main() {
    NODE root = NULL;
    int n, i, arr[50];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        root = insert(root, arr[i]);
    }

    printf("\nSorted elements using Binary Tree Sort:\n");
    inorder(root);

    return 0;
}

```

Output:

```

Enter number of elements: 6
Enter 6 elements:
45 12 78 34 23 56

```

```

Sorted elements using Binary Tree Sort:
12 23 34 45 56 78

```

4. Write a C program to sort an array of n elements using Heap sort technique.


```
#include <stdio.h>

/* Swap helper */
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

/* Heapify subtree rooted at index i in array a[1..n] */
void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i;
    int right = 2 * i + 1;

    if (left <= n && a[left] > a[largest])
        largest = left;

    if (right <= n && a[right] > a[largest])
        largest = right;

    if (largest != i) {
        swap(&a[i], &a[largest]);
        heapify(a, n, largest);
    }
}
```

```
}
```

```
/* Build max heap */
```

```
void buildHeap(int a[], int n) {
```

```
    int i;
```

```
    for (i = n / 2; i >= 1; i--)
```

```
        heapify(a, n, i);
```

```
}
```

```
/* Heap sort */
```

```
void heapSort(int a[], int n) {
```

```
    int i;
```

```
    buildHeap(a, n);
```

```
    for (i = n; i >= 2; i--) {
```

```
        swap(&a[1], &a[i]); // move current max to end
```

```
        heapify(a, i - 1, 1); // restore heap on reduced array
```

```
    }
```

```
}
```

```
void main() {
```

```
    int n, i;
```

```
    int a[100];
```

```
    printf("Enter number of elements: ");
```

```
scanf("%d", &n);

printf("Enter %d elements:\n", n);

for (i = 1; i <= n; i++)
    scanf("%d", &a[i]);

heapSort(a, n);

printf("\nSorted elements using Heap Sort:\n");

for (i = 1; i <= n; i++)
    printf("%d\t", a[i]);

printf("\n");
}
```

Output:

Enter number of elements: 6

Enter 6 elements:

40 10 50 20 30 60

Sorted elements using Heap Sort:

10 20 30 40 50 60